UKIEPC Team Reference Document — Python (PyPy 3.9, Kattis)

§		§		§		§	
1	Template, I/O, testing	5	Prefix sums, two pointers	9	Trees, LCA, linked list	13	Number theory, counting
2	Python stdlib	6	Greedy	10	Fenwick, segment trees	14	Geometry
3	Everyday building blocks	7	Graphs, BFS/DFS, DSU	11	Dynamic programming	15	Flow, matching, SCC, 2-SAT
4	Binary/ternary search	8	Shortest paths, toposort, MST	12	Strings	16	Bits, meet-in-middle, stress

Names used everywhere (the standard competitive-programming vocabulary — tutorials, editorials and this document all speak it): `n`, `m` = number of elements / of edges · `a` = the input array · `u`, `v` = two graph nodes, `w` = an edge weight · `adj` = adjacency list · `r`, `c` = row, column · `lo`, `hi` = ends of a search range · `l`, `r` = ends of an array range · `dp` = the DP table · `q` = queue, `dq` = deque, `pq` = heap (priority queue) · `s`, `t` = a string and a pattern, or source and sink in flow · `INF = 1 << 60` · `MOD = 10**9 + 7`.

1. Template and I/O

```
import sys
from collections import deque, defaultdict, Counter
import heapq, bisect, math

data = sys.stdin.buffer.read().split()   # every token in
the input, as bytes
it = iter(data)
def nxt():
    return next(it)                      # the next token

def main():
    n = int(nxt())
    arr = [int(nxt()) for _ in range(n)]
    out = []
    # ... solve, append each output line to out ...
    print("\n".join(map(str, out)))

main()
```

I/O variants.

```
input = sys.stdin.readline        # fast line reads; keeps
'\n' -> .strip() strings!
n, m = map(int, input().split())
a = list(map(int, input().split()))

for line in sys.stdin:            # read until end of input
(EOF)
    ...

s = nxt().decode()                # tokens from buffer are
BYTES; decode for strings

print(f"{x:.6f}")                 # fixed decimals; NEVER
print extra text
print(*row)                       # space-separated list
# interactive problems only: print(ans, flush=True)
```

The input shapes you'll meet (all using `nxt()` from the template — it doesn't care about line breaks, only token order):

```
# shape 1: "first line T, then T independent test cases"
t = int(nxt())
for _ in range(t):
    solve()                       # CAREFUL: reset ALL state
inside solve()

# shape 2: n, then n numbers (one line or many — nxt()
doesn't care)
n = int(nxt())
a = [int(nxt()) for _ in range(n)]

# shape 3: a grid of characters, H rows of width W
H, W = int(nxt()), int(nxt())
g = [list(nxt().decode()) for _ in range(H)]      # g[row]
[col]

# shape 4: mixed tokens per record, e.g. "alice 3 7" times n
for _ in range(n):
    name = nxt().decode()
    x, y = int(nxt()), int(nxt())

# shape 5: "read until end of input" (no count given) —
line-based instead:
for line in sys.stdin:
    if not line.strip():
        continue                  # skip blank lines
    ...
```

Formatting: `f"{x:06d}"` zero-pad, `f"{x:b}"`/`:o`/`:x` bases, `f"{s:>10}"` right-align, `f"{x:.3e}"` scientific. Note `round()` is banker's rounding: `round(2.5) == 2`; for half-up use `math.floor(x + 0.5)`.

Judge gotcha. stdout is compared (usually) exactly: match newlines, no trailing spaces, no "Answer:" prefixes unless asked. Case numbers: `print(f"Case #{t}: {ans}")` — copy the format character-for-character.

Testing before you submit. Paste each sample into a file and run against it — never retype input by hand:

```
pypy3 a.py < 1.in                 # run on the sample
(python3 if no pypy3)
pypy3 a.py < 1.in > 1.out
diff 1.out 1.ans                  # byte-exact check vs
the expected output
```

If samples pass but the judge says WA, write a brute force and stress-test (section 16).

2. Python standard library

Everything here exists in Python 3.9.2 / PyPy 7.3 — the judge's version. Each line shows what you get back on a small example.

collections.

```
c = Counter("aab")            # Counter({'a': 2, 'b': 1})
c['z']                        # 0 — missing key is 0, never
a KeyError
c.most_common(2)              # [('a', 2), ('b', 1)] —
biggest counts first
c.most_common()               # all of them, sorted by count
descending
c1 + c2                       # add counts;  c1 & c2 =
pairwise min;  c1 | c2 = max
sum(c.values())               # total number of items
counted

d = defaultdict(list)         # missing key -> [] created on
the spot
d['k'].append(3)              # no need to check 'k' exists
first
d = defaultdict(int)          # missing key -> 0: d['k'] +=
1 just works
d = defaultdict(set)          # missing key -> set():
d['k'].add(v)
# CAREFUL: just READING d[k] inserts k. To test membership
use 'k in d',
# never 'if d[k]' — that creates the key and can grow the
dict mid-loop.

dq = deque([1, 2, 3])         # double-ended queue
dq.append(4); dq.appendleft(0)      # O(1) at BOTH ends
(lists are O(n) at the front)
dq.pop(); dq.popleft()              # -> 4 and 0
dq[0], dq[-1]                       # peek both ends without
removing
dq = deque(maxlen=k)          # append beyond k silently
drops the far end:
                              # a ready-made sliding window
of the last k items
```

heapq — a min-heap stored in a plain list (`h[0]` is always the smallest). There is no decrease-key: push a duplicate instead and skip stale entries when popping (see Dijkstra, section 8).

```
h = [5, 1, 4]
heapq.heapify(h)              # rearranges in place to a
valid heap, O(n); h[0] == 1
heapq.heappush(h, 2)          # add one value, O(log n)
heapq.heappop(h)              # -> 1, removes it, O(log n)
heapq.heappush(h, (dist, node))     # tuples compare by
first item, then second —
d, u = heapq.heappop(h)             # so (key, payload) pops
smallest key first
heapq.nlargest(3, a)          # the 3 biggest of a, sorted;
also nsmallest; key= OK
```

```
# max-heap trick: push -x, negate when popping. For tuples:
push (-key, item).
```

bisect — binary search in a SORTED list, O(log n). The two functions differ only at values already present. NOTE: `bisect(..., key=...)` does NOT exist in 3.9 — to search by a field, keep a separate list of keys: `ks = [k for k, v in pairs]`.

```
a = [10, 20, 20, 30]
bisect.bisect_left(a, 20)     # -> 1: first index i with
a[i] >= 20  (lower_bound)
bisect.bisect_right(a, 20)    # -> 3: first index j with
a[j] >  20  (upper_bound)
bisect.bisect_left(a, 25)     # -> 3: where 25 WOULD be
inserted
# how many values equal x:
count = bisect.bisect_right(a, x) - bisect.bisect_left(a, x)
# is x present at all:
i = bisect.bisect_left(a, x)
present = i < len(a) and a[i] == x
# how many values < x: bisect_left;   how many <= x:
bisect_right
bisect.insort(a, 25)          # insert keeping order — O(n)
shift! fine for n <= ~1e5
```

itertools.

```
from itertools import permutations, combinations, product,
accumulate, groupby
permutations([1,2,3])         # (1,2,3) (1,3,2) (2,1,3) ...
— all 3! = 6 orderings
permutations(a, r)            # orderings of length r only
combinations([1,2,3], 2)      # (1,2) (1,3) (2,3) — subsets:
no repeats, no reorder
product([0,1], repeat=3)      # (0,0,0) (0,0,1) ... (1,1,1)
— k nested loops in one
product(xs, ys)               # every pair (x, y) — like a
double loop
list(accumulate([3,1,4]))             # [3, 4, 8]    running
sums
list(accumulate([3,1,4], initial=0))  # [0, 3, 4, 8] prefix
sums, length n+1
for key, group in groupby("aabbbc"):  # runs of equal
ADJACENT items:
    print(key, len(list(group)))      # a 2 / b 3 / c 1 —
sort first to group globally
```

math and built-in pow.

```
math.gcd(12, 18, 24)          # 6;  3.9+ accepts any number
of args: gcd(*nums)
math.lcm(4, 6)                # 12; math.lcm is new in 3.9 —
safe on this judge
math.isqrt(10**18)            # exact integer sqrt, floor —
int(n**0.5) is WRONG
                              # for big n (float rounding);
isqrt never is
math.comb(10, 3)              # 120, exact binomial;
math.factorial(n) too
math.hypot(3, 4)              # 5.0 = sqrt(3*3 + 4*4)
math.atan2(y, x)              # angle of point (x,y) in (-
pi, pi] — handles x=0 fine
math.ceil(a / b)              # or integer-only: -(-a // b)
== (a + b - 1) // b
pow(a, b, m)                  # a^b mod m in O(log b) —
never a**b % m for big b
pow(a, -1, m)                 # modular inverse of a mod m;
needs gcd(a, m) == 1
```

functools.

```
from functools import lru_cache, cmp_to_key, reduce
@lru_cache(maxsize=None)      # memoise by arguments — turns
plain recursion into DP
def f(i, j): ...              # args must be hashable;
f.cache_clear() between cases!
                              # (recursion depth still
applies — fine for digit DP)
reduce(math.gcd, nums)        # fold a list to one value:
gcd of everything
```

strings. Strings are immutable — every "change" builds a new string.

```
"  hi \n".strip()             # "hi" — strips whitespace
both ends; also lstrip/rstrip
"a b  c".split()              # ['a','b','c'] — any
whitespace, no empty pieces
"a,,b".split(',')             # ['a','','b'] — exact
separator, KEEPS empty pieces
" ".join(["a","b"])           # "a b" — join only accepts
strings: map(str, nums)
"aBc".lower(); "aBc".upper()  # "abc", "ABC"; also
s.swapcase()
"abcab".find("ab")            # 0 — first index, -1 if
absent (index() raises instead)
"abcab".count("ab")           # 2 — non-overlapping
"aaa".replace("a", "b")       # "bbb" — ALL occurrences, new
string
s.startswith(p); s.endswith(p)      # also accept a tuple of
options
"42".isdigit(); "ab".isalpha()      # character-class tests
ord('a'), chr(97)             # 97, 'a';  letter index:
ord(ch) - ord('a')
import string
string.ascii_lowercase        #
'abcdefghijklmnopqrstuvwxyz'; also digits, uppercase
chars = list(s)               # to edit: list out, change
cells, "".join(chars) back
s[::-1]                       # reversed copy;  s == s[::-1]
= palindrome test
s[2:5]                        # slice, half-open: indices
2,3,4; works on lists too
int('ff', 16)                 # 255 — parse any base 2..36;
int('101', 2) == 5
"abc" < "abd"                 # strings compare
lexicographically — sorting words
                              # "just works", but "10" < "9"
as strings! (sort ints)
```

Sorting.

```
a.sort()                      # in place, O(n log n);
sorted(a) returns a new list
a.sort(key=lambda t: t[1])    # sort by one field
a.sort(key=lambda t: (-t[1], t[0]))   # score DESCENDING,
ties by name ascending —
                              # negate numbers for
descending; strings can't be
                              # negated: sort twice instead
(stable, see below)
a.sort(reverse=True)          # everything descending
a.sort(key=len)               # any function of the element
works
# sort is STABLE: equal keys keep their input order. So
multi-key sorts can be
# done in passes: sort by the SECONDARY key first, then by
the primary.
from functools import cmp_to_key      # a real comparator,
if a key is impossible:
a.sort(key=cmp_to_key(compare))       # compare(x, y) ->
negative / 0 / positive
max(a, key=lambda t: t[1])    # min/max also accept key=
# coordinate compression (shrink big values to 0..k-1
keeping their order):
vals = sorted(set(a))                 # distinct values,
ascending
rank = {v: i for i, v in enumerate(vals)} # value -> 0-based
rank
b = [rank[x] for x in a]              # e.g. [100, 7,
100] -> [1, 0, 1]
```

Gotchas (PyPy 3.9 on Kattis).

- `print()` per line in a big loop is slow — collect lines and write once. `input()` is slow — alias `sys.stdin.readline` or read everything at the start.
- Recursion is slow and stack-limited: write BFS/DFS **iteratively**. If forced: `sys.setrecursionlimit(1 << 25)` and hope.
- `[[0]*W for _ in range(H)]`, **never** `[[0]*W]*H` (all rows become the same list).
- Integer division floors: `-7 // 2 == -4`. But `int(-3.5) == -3` truncates. `%` is always non-negative for a positive modulus: `(-1) % 5 == 4` (handy for mod maths, unlike C++).
- No `match`, no `int.bit_count()` (use `bin(x).count('1')`), no `zip(strict=)` in 3.9. `math.lcm`, `math.isqrt`, `math.comb`, `pow(a, -1, m)` all exist.
- Prefer the int sentinel `INF = 1 << 60` to `float('inf')` (keeps everything integer).
- Copy vs alias: `b = a` is the SAME list; copy with `b = list(a)`; copy a grid row by row.
- Closures capture variables late: `lambda x, k=k: ...` to freeze a loop variable.
- numpy is **not** guaranteed on the judge. Don't rely on it.

3. Everyday building blocks

Small idioms used inside almost every solution. Each shown on a concrete example.

Slices — grabbing parts of a list (all of these also work on strings):

```
a = [10, 20, 30, 40, 50]
a[-1], a[-2]            # 50, 40 — negative index counts
```

```
from the end
a[:3]                   # [10, 20, 30]  the first three
a[-2:]                  # [40, 50]      the last two
a[1:4]                  # [20, 30, 40]  half-open: start
included, end excluded
a[::2]                  # [10, 30, 50]  every second element
a[::-1]                 # [50, 40, 30, 20, 10]  reversed
copy
b = a[:]                # a copy (any slice copies; b = a
would alias!)
```

Walking lists together — enumerate and zip:

```
for i, x in enumerate(a):        # index AND value: 0 10, 1
20, ...
    ...
for x, y in zip(xs, ys):         # two lists in lockstep
(stops at the shorter)
    ...
pairs = list(zip(names, scores)) # glue columns into rows:
[(n0,s0), (n1,s1), ...]
names, scores = zip(*pairs)      # and apart again ("unzip")
for prev, cur in zip(a, a[1:]):  # every ADJACENT pair —
differences, rises/falls
    ...
is_sorted = all(x <= y for x, y in zip(a, a[1:]))
```

Counting, grouping, membership:

```
count = sum(1 for x in a if x > 0)   # count matching (True
counts as 1)
any(x < 0 for x in a)                # is there at least
one? (stops early)
all(x >= 0 for x in a)               # do all satisfy it?

groups = defaultdict(list)           # group words that are
anagrams:
for w in words:
    groups["".join(sorted(w))].append(w)   # same letters ->
same key

seen = set()                         # de-duplicate KEEPING
first-seen order
out = []
for x in a:
    if x not in seen:                # set membership is
O(1); on a list it's O(n)
        seen.add(x)
        out.append(x)
```

min/max beyond the obvious:

```
max(a, key=len)                      # the LONGEST element,
not the largest
max(a, default=0)                    # no crash on an empty
list
best_i = max(range(n), key=lambda i: a[i])   # ARGmax: INDEX
of the biggest
idx = sorted(range(n), key=lambda i: a[i])   # argsort:
indices ordered by value
rank = {v: i for i, v in enumerate(sorted(set(a)))}   #
value -> 0-based rank
```

Merge overlapping intervals (constant easy-problem ingredient):

```
ivs.sort()                           # by start
merged = []
for s, e in ivs:
    if merged and s <= merged[-1][1]:            # overlaps
the last kept one
        merged[-1][1] = max(merged[-1][1], e)    # extend it
    else:
        merged.append([s, e])                    # start a
new block
# [1,4],[2,6],[8,9] -> [1,6],[8,9];  use s < merged[-1][1]
if touching is NOT overlap
```

Run-length encoding — compress runs, or iterate them:

```
from itertools import groupby
[(ch, len(list(g))) for ch, g in groupby("aaabbc")]   #
[('a',3), ('b',2), ('c',1)]
```

2D grids — the cookbook:

```
g = [list(nxt().decode()) for _ in range(H)]     # read H
rows of characters
g = [[int(nxt()) for _ in range(W)] for _ in range(H)]   #
... or of numbers
g[r][c]                              # ROW first, then
column
g2 = [row[:] for row in g]           # copy row by row
(never g2 = g)
rows = ["".join(row) for row in g]   # back to strings for
printing
t = [list(col) for col in zip(*g)]           # transpose
(flip rows<->columns)
rot = [list(col) for col in zip(*g[::-1])]   # rotate 90
degrees clockwise
flat = [x for row in g for x in row]         # flatten to
one list
```

Small but constant:

```
q, r = divmod(17, 5)                 # quotient and
remainder at once: 3, 2
a, b = b, a                          # swap, no temp
variable
if 0 <= r < H and 0 <= c < W: ...    # chained comparison =
bounds check
x = y if cond else z                 # inline if
INF = 1 << 60                        # "infinity" that stays
an integer
letters = [0] * 26                   # letter counts:
letters[ord(ch) - 97] += 1
```

Round-robin simulation with a deque ("every k-th person leaves the circle"):

```
dq = deque(range(n))                 # people 0..n-1 in a
circle
while len(dq) > 1:
    dq.rotate(-(k - 1))              # k-1 people step from
the front to the back
    dq.popleft()                     # the k-th is
eliminated
survivor = dq[0]
```

4. Binary and ternary search

Spot it. "Smallest/largest x such that…", "minimise the maximum…", "maximise the minimum…", or: checking a GIVEN answer is easy but finding the best one isn't → binary search on the answer. A huge range (up to 10^{18}) with a yes/no question inside is the giveaway. "Best single peak of a rise-then-fall function" → ternary search.

Binary search needs ONE thing: an answer space where `ok(x)` flips once and never flips back (monotone). Each step throws away the half that cannot contain the boundary, so it takes O(log(hi−lo)) calls of `ok`.

```
def first_true(lo, hi, ok):
    # ok(x) looks like  F F F F T T T  on lo..hi (false,
then true forever).
    # Returns the position of the FIRST True, or hi+1 if ok
is never true.
    hi += 1                          # allow "no true exists"
as the answer hi+1
    while lo < hi:                   # invariant: answer is
always in [lo, hi]
        mid = (lo + hi) // 2
        if ok(mid):
            hi = mid                 # mid works -> the first
true is mid or earlier
        else:
            lo = mid + 1             # mid fails -> the first
true is after mid
    return lo                        # lo == hi == the
boundary

def last_true(lo, hi, ok):
    # Mirror image: ok(x) looks like  T T T F F F.
    # Returns the position of the LAST True, or lo-1 if ok
is never true.
    lo -= 1
    while lo < hi:
        mid = (lo + hi + 1) // 2     # round UP: without +1,
lo = mid could loop
                                     # forever when hi == lo
+ 1
        if ok(mid):
            lo = mid                 # mid works -> the last
true is mid or later
        else:
            hi = mid - 1             # mid fails -> the last
true is before mid
    return lo
```

"Minimise the maximum…", "maximise the minimum…", "smallest x such that…" → binary search on the answer: write `ok(x)` as a greedy/linear check, confirm it is monotone, search. Real-valued: loop a fixed 100 times instead of testing a tolerance.

```
lo, hi = 0.0, 1e18                   # real-valued search:
same idea, but a fixed
for _ in range(100):                 # iteration count
instead of lo < hi
    mid = (lo + hi) / 2              # (100 halvings is
```

```
plenty of precision and
    if ok(mid):                         # can't loop forever on
float rounding)
        hi = mid
    else:
        lo = mid
# answer: lo (or hi — they're essentially equal now)
```

Ternary search — for a function that rises then falls (strictly unimodal), find the peak. Compare two interior points: the side with the smaller value cannot contain the peak, drop it.

```
lo, hi = L, R                           # integer version
while hi - lo > 2:                      # stop when 3 or fewer
candidates remain
    m1 = lo + (hi - lo) // 3            # two probes, one third
in from each end
    m2 = hi - (hi - lo) // 3
    if f(m1) < f(m2):
        lo = m1 + 1                     # peak can't be at m1
or left of it
    else:
        hi = m2 - 1                     # peak can't be at m2
or right of it
best = max(f(x) for x in range(lo, hi + 1))   # check the
few survivors directly
# real version: fixed 200 iterations, keep the third with
the better endpoint
# minimum instead of maximum: flip the comparison
```

5. Prefix sums, difference arrays, two pointers

Spot it. "Answer q range-sum questions", "apply many +v-on-a-range updates then print the array", "longest/shortest subarray with ≤ k of something" (sliding window), "find a pair summing to k in a sorted array", "maximum of every window of size k". All share one smell: a naive double loop over ranges that's too slow.

The common theme: pay O(n) once up front so that every later question costs O(1), instead of re-scanning the array per question. **Prefix sums** answer "sum of any range?"; a **difference array** is the same idea backwards, batching "add v to a whole range" updates; **two pointers** replace a double loop with one sweep where both ends only move forward.

```
# prefix[i] = sum of the FIRST i elements. A range sum is
then a difference:
# sum(a[l..r]) = (a[0]+...+a[r]) - (a[0]+...+a[l-1]) =
prefix[r+1] - prefix[l]
from itertools import accumulate
prefix = list(accumulate(a, initial=0))   # a = [3,1,4] ->
prefix = [0, 3, 4, 8]

def range_sum(l, r):                    # sum of a[l..r], 0-
indexed inclusive, O(1)
    return prefix[r + 1] - prefix[l]
# works for XOR too (its own inverse): prefix_xor[r+1] ^
prefix_xor[l]
```

2D prefix sums — same trick for rectangle sums in a grid.

```
# P[i][j] = sum of the whole sub-grid above-and-left of (i,
j), exclusive.
# Padding row/column 0 with zeros avoids every boundary
special-case.
P = [[0] * (W + 1) for _ in range(H + 1)]
for i in range(H):
    for j in range(W):
        # cell + sum above + sum left - overlap (counted
twice)
        P[i+1][j+1] = grid[i][j] + P[i][j+1] + P[i+1][j] -
P[i][j]

def rect_sum(r1, c1, r2, c2):           # sum over the
rectangle, inclusive corners
    # big rect - strip above - strip left + top-left corner
(subtracted twice)
    return P[r2+1][c2+1] - P[r1][c2+1] - P[r2+1][c1] + P[r1]
[c1]
```

Difference array — many "add v to every element in [l, r]" updates, all values read once at the END. Each update is O(1); one prefix-sum pass at the end materialises the array.

```
diff = [0] * (n + 1)                    # diff[i] = how much
the value changes at index i
# each update "add v on [l, r]" becomes two marks:
diff[l] += v                            # from l onward,
everything is +v ...
diff[r + 1] -= v                        # ... until r+1, where
the +v stops
a = list(accumulate(diff))[:n]          # running total of diff
= the final values
# 2D version: +v at (r1,c1), -v at (r1,c2+1) and (r2+1,c1),
+v at (r2+1,c2+1),
# then prefix-sum rows and columns
```

Two pointers / sliding window (needs monotonicity, e.g. all a[i] ≥ 0):

```
best = 0
window_sum = 0
l = 0
for r in range(n):                      # longest window with
sum <= k
    window_sum += a[r]                  # grow the window to
the right ...
    while window_sum > k:               # ... shrink from the
left while it's invalid
        window_sum -= a[l]
        l += 1
    best = max(best, r - l + 1)         # window a[l..r] is the
longest valid one ending at r
# O(n) total: l and r each move forward at most n times.
# opposite ends on a SORTED array (two-sum): l, r = 0, n-1;
move the side that helps
```

Sliding window maximum (monotonic deque) and **next greater element** (monotonic stack):

```
dq = deque()          # indices; values decrease from front
to back
result = []
for i in range(n):    # maximum of every window of size k
    while dq and a[dq[-1]] <= a[i]:
        dq.pop()      # a[i] makes older smaller values
useless
    dq.append(i)
    if dq[0] <= i - k:
        dq.popleft()  # front fell out of the window
    if i >= k - 1:
        result.append(a[dq[0]])
```

```
nge = [-1] * n        # index of the next strictly greater
element to the right
stack = []
for i in range(n - 1, -1, -1):
    while stack and a[stack[-1]] <= a[i]:
        stack.pop()
    if stack:
        nge[i] = stack[-1]
    stack.append(i)
# same trick solves: largest rectangle in a histogram, stock
span, closest smaller
```

6. Greedy

Spot it. Scheduling/intervals/deadlines, "minimum number of X to cover Y", problems where sorting by SOME key makes each decision obviously safe, and huge constraints ($n \geq 10^5$ with no structure for DP) — if the intended solution is clearly O(n log n), it's usually sort + greedy or sort + two pointers.

State the exchange argument in one sentence before coding ("swapping two adjacent choices never improves the result because …"). If you can't, it's probably DP. Cheap safety net: brute-force small random cases and compare (section 16).

```
# Interval scheduling: pick the MAXIMUM number of non-
overlapping intervals.
# Greedy rule: always take the interval that ENDS first — it
leaves the most
# room for the rest. (Sorting by start or by length is
WRONG: one long early
# interval would block many short ones.)
events.sort(key=lambda e: e[1])       # sort by end time
last_end = -INF                       # when the last taken
interval finished
count = 0
for s, e in events:
    if s >= last_end:                 # starts after the
previous one ended ->
        count += 1                    # take it (change to s
> last_end if
        last_end = e                  # touching endpoints
count as overlap)
```

```
# Minimum rooms/platforms so all intervals fit = the busiest
moment.
# Turn each interval into two events: +1 person arrives, -1
person leaves,
# then sweep through time counting how many are present.
times = []
for s, e in intervals:
    times.append((s, 1))              # at time s: one more
```

```
running
    times.append((e, -1))           # at time e: one fewer
times.sort()          # tuples sort by time, then by delta,
so at equal times
                      # -1 comes before +1: an interval
ending frees its room
                      # for one starting at the same moment
current = best = 0
for _, delta in times:
    current += delta                # how many intervals
are open right now
    best = max(best, current)       # the peak = rooms
needed
```

Other classics: minimise lateness → sort by deadline; fractional knapsack → sort by value/weight descending; pair smallest with largest → sort + two pointers from both ends. Classic greedy **failures**: coin change with coins {1,3,4}, target 6 (greedy 4+1+1, optimal 3+3) → DP; 0/1 knapsack by ratio → DP.

7. Graphs: traversal, components, DSU

Spot it. Anything with pairwise connections: cities and roads, grid mazes, prerequisites, friendships, "is X reachable from Y", "how many groups/islands". Also problems with NO graph in sight but states and moves — a lock with buttons, a word changed one letter at a time: states are nodes, legal moves are edges (an *implicit* graph), and BFS gives the fewest moves. "Are these in the same group" with things merging over time → DSU.

Build: adjacency list. `adj[u]` is a plain Python list holding the neighbours of node u. This is the default representation for every algorithm in this document.

```
adj = [[] for _ in range(n)]   # n empty lists — NOT [[]] *
n (that's n aliases
                               # of ONE list; appending to
adj[0] would change all)
for _ in range(m):             # m = number of edges
    u = int(nxt()) - 1         # input numbers nodes 1..n;
we use 0..n-1
    v = int(nxt()) - 1         #   (skip the -1 if input is
already 0-indexed)
    adj[u].append(v)           # u has neighbour v ...
    adj[v].append(u)           # ... and v has neighbour u
(undirected edge).
                               # DIRECTED edge u -> v: only
the first append.
# weighted graph: store (neighbour, weight) pairs instead of
bare neighbours:
#   adj[u].append((v, w))      then loops become: for v, w
in adj[u]
# example: n=4, edges (0,1) (0,2) (2,3)  ->  adj = [[1,2],
[0], [0,3], [2]]
```

Alternative: **adjacency matrix** — `mat[u][v]` = weight of edge u→v (or INF if absent). O(n²) memory and O(n) to list one node's neighbours, so only for small dense graphs (n ≤ ~500) or when you need "is there an edge u–v?" in O(1). Floyd–Warshall (section 8) uses it.

```
mat = [[INF] * n for _ in range(n)]
for i in range(n):
    mat[i][i] = 0              # a node reaches itself for
free
for _ in range(m):
    u, v, w = int(nxt()) - 1, int(nxt()) - 1, int(nxt())
    mat[u][v] = min(mat[u][v], w)    # min() keeps the
cheapest duplicate edge
    mat[v][u] = min(mat[v][u], w)    # drop this line for a
directed graph
```

BFS. The simplest version — just visit everything reachable from `start`, nearest first:

```
seen = [False] * n
seen[start] = True
q = deque([start])
while q:
    u = q.popleft()            # FIFO: oldest first = level
by level
    for v in adj[u]:
        if not seen[v]:
            seen[v] = True     # mark when PUSHING, not when
popping —
            q.append(v)        # or the same node enters the
queue many times
```

That's all BFS is. Because it visits in level order, recording *which level* gives shortest paths in unweighted graphs — the full version below just swaps `seen` for `dist` (–1 = not seen yet) and remembers each node's `parent` so the path can be reconstructed:

```
def bfs(start):
    dist = [-1] * n
    parent = [-1] * n
    dist[start] = 0
    q = deque([start])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if dist[v] == -1:           # not seen yet
                dist[v] = dist[u] + 1   # one level deeper
than u
                parent[v] = u           # we reached v
through u
                q.append(v)
    return dist, parent
# path: follow parent[] from t back to s, then reverse;
check dist[t] != -1 first!
# multi-source ("nearest fire"): put ALL sources in q with
dist 0 before the loop
```

Grid BFS.

```
DIRS = ((1,0), (-1,0), (0,1), (0,-1))    # add diagonals for
8-direction problems
dist = [[-1] * W for _ in range(H)]
dist[sr][sc] = 0
q = deque([(sr, sc)])
while q:
    r, c = q.popleft()
    for dr, dc in DIRS:
        nr, nc = r + dr, c + dc
        if 0 <= nr < H and 0 <= nc < W and grid[nr][nc] !=
'#' and dist[nr][nc] == -1:
            dist[nr][nc] = dist[r][c] + 1
            q.append((nr, nc))
# knight moves: ((1,2),(2,1),(-1,2),(-2,1),(1,-2),(2,-1),
(-1,-2),(-2,-1))
# always check nr >= 0 explicitly: grid[-1] silently wraps
around in Python!
```

BFS works on any state space, not only grids: (position, keys held), (vertex, parity), lock-dial combinations… State = a tuple, neighbours = legal moves, use a dict for distances.

0–1 BFS — edge weights 0 or 1 only, O(V+E): a deque; weight-0 edges go to the front.

```
dist = [INF] * n
dist[s] = 0
dq = deque([s])
while dq:
    u = dq.popleft()
    for v, w in adj[u]:                # w is 0 or 1
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            if w == 0:
                dq.appendleft(v)
            else:
                dq.append(v)
```

Iterative DFS. For plain reachability / components:

```
seen = [False] * n
components = 0
for s in range(n):
    if seen[s]:
        continue
    components += 1
    seen[s] = True
    stack = [s]
    while stack:
        u = stack.pop()
        for v in adj[u]:
            if not seen[v]:
                seen[v] = True
                stack.append(v)
```

When you need exit order (post-order: for tree DP, SCC, toposort by DFS):

```
stack = [(s, 0)]       # (vertex, index of the next
neighbour to try)
seen[s] = True
post = []
while stack:
    u, i = stack[-1]
    if i < len(adj[u]):
        stack[-1] = (u, i + 1)
        v = adj[u][i]
        if not seen[v]:
            seen[v] = True
            stack.append((v, 0))
```

```
    else:
        post.append(u)  # u is finished: all its children
are done
        stack.pop()
```

Bipartite check / 2-colouring (BFS):

```
colour = [-1] * n
ok = True
for s in range(n):
    if colour[s] != -1:
        continue
    colour[s] = 0
    q = deque([s])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if colour[v] == -1:
                colour[v] = colour[u] ^ 1
                q.append(v)
            elif colour[v] == colour[u]:
                ok = False
```

Cycle detection. Undirected: during BFS/DFS, an edge to an already-visited vertex that isn't the parent → cycle (or: DSU `union` returns False). Directed: Kahn's toposort output has fewer than n vertices → cycle.

DSU (union–find) — maintains "which group is each element in" while groups merge. Each group is a little tree of pointers; `find` walks up to the group's representative (the root), `union` hangs one root under the other. Two tricks keep every walk near O(1): path compression (`find` re-points nodes it passes straight at the root) and always hanging the smaller group under the bigger. Iterative find (PyPy-safe):

```
parent = list(range(n))
size = [1] * n

def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]   # path compression
        x = parent[x]
    return x

def union(a, b):
    a = find(a)
    b = find(b)
    if a == b:
        return False                    # already in the
same component
    if size[a] < size[b]:
        a, b = b, a
    parent[b] = a
    size[a] += size[b]
    return True
# component size: size[find(x)].  No deletions: process
offline / in reverse time.
```

8. Shortest paths, toposort, MST

Spot it / which algorithm. "Cheapest/fastest route" with edge costs. Decision list: all edges cost 1 → BFS (section 7); costs ≥ 0 → Dijkstra; negative costs or "detect a profitable cycle" → Bellman–Ford; all-pairs distances and n ≤ ~400 → Floyd–Warshall; dependencies/"must do X before Y" → toposort (+ DP on the DAG); "connect everything as cheaply as possible" → MST.

Dijkstra — non-negative weights only, O((V+E) log V). Why it works: always settle the unvisited node with the smallest known distance — with no negative edges, no later route can undercut it, so once popped a node is final. The heap serves nodes in exactly that order.

```
def dijkstra(adj, s):
    dist = [INF] * n
    parent = [-1] * n
    dist[s] = 0
    pq = [(0, s)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue        # stale heap entry — this check
is REQUIRED for speed
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                parent[v] = u
                heapq.heappush(pq, (d + w, v))
    return dist, parent
# variants: several sources -> seed them all; stop early ->
return when t is popped;
# grid version: treat cell (r, c) as node id r*W + c;
# maximise a product of probabilities -> minimise sum of -
log(p)
```

Bellman–Ford — negative edges OK, O(VE). A relaxation in the n-th round → negative cycle.

```
dist = [INF] * n
dist[s] = 0
for round_no in range(n):
    changed = False
    for u, v, w in edges:
        if dist[u] < INF and dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            changed = True
    if not changed:
        break
has_negative_cycle = changed            # still True after n
full rounds
```

Floyd–Warshall — all pairs, negative edges OK (but no negative cycles), O(V³), V ≤ ~400 in PyPy. The outer loop **must** be k.

```
dist = [[INF] * n for _ in range(n)]    # dist[i][j] = best
known i -> j
for i in range(n):
    dist[i][i] = 0
for u, v, w in edges:
    dist[u][v] = min(dist[u][v], w)     # min() handles
duplicate edges
for k in range(n):                      # "routes may now
ALSO pass through k"
    for i in range(n):
        for j in range(n):
            if dist[i][k] + dist[k][j] < dist[i][j]:
                dist[i][j] = dist[i][k] + dist[k][j]
# afterwards dist[i][i] < 0 means a negative cycle passes
through i
```

Topological sort (Kahn) — DAG ordering, cycle detection, basis for DAG DP.

```
indeg = [0] * n
for u in range(n):
    for v in adj[u]:
        indeg[v] += 1
q = deque(u for u in range(n) if indeg[u] == 0)   # use
heapq for lexicographic order
order = []
while q:
    u = q.popleft()
    order.append(u)
    for v in adj[u]:
        indeg[v] -= 1
        if indeg[v] == 0:
            q.append(v)
# len(order) < n  ->  the graph has a cycle
```

DP on a DAG (process vertices in topological order):

```
longest = [0] * n          # longest path (in edges) ending
at v
ways = [0] * n             # number of distinct paths from s
ways[s] = 1
for u in order:
    for v in adj[u]:
        longest[v] = max(longest[v], longest[u] + 1)
        ways[v] = (ways[v] + ways[u]) % MOD
```

MST — Kruskal (sort edges + DSU). Prim with a heap is the same idea keyed by cheapest edge into the tree.

```
edges.sort()               # list of (w, u, v)
total = 0
used = 0
for w, u, v in edges:
    if union(u, v):
        total += w
        used += 1
        if used == n - 1:
            break
# used < n-1 -> graph is disconnected.  Maximum spanning
tree: sort descending.
# "minimise the maximum edge on a path" -> the max edge on
the MST path
```

9. Trees

Spot it. The statement says "n nodes and n−1 edges, connected", "exactly one path between any two", a file-system / company hierarchy, or "a network with no redundant links". Common asks: distance between nodes (LCA), something per subtree (bottom-up sweep), longest path (diameter).

What a tree is. A connected graph with n nodes and exactly n−1 edges. No cycles, so between any two nodes there is exactly ONE path. A "rooted tree" is the same thing plus a chosen root: every node except the root then has one *parent* (its neighbour on the path toward the root) and the rest of its neighbours are its *children*.

How contest trees differ from CS-course trees. In a course, a tree is `Node` objects with pointers and `insert`/`delete` methods (binary search trees etc.). In a contest, the problem hands you the finished tree in the input as a list of edges, and it NEVER changes — there is no inserting or removing, no Node class, no pointers. You read the edges into an adjacency list once, then only walk it and compute on it. (If a problem needs a *dynamic* sorted structure — insert values, delete values, query — that is not a tree problem: use the substitutes in section 10, "No sorted multiset". If it needs insert/remove *at a position*, see the linked list at the end of this section.)

How to store it: adjacency list. `adj[u]` is the list of u's neighbours. Build it by appending each edge to both endpoints' lists (an undirected edge goes both ways). This is all a tree is in memory — a list of lists. The example tree below is used for the whole section:

```
#  the example tree        its edges: (0,1) (0,2) (2,3)
(2,4)
#        0
#       / \               adj[0] = [1, 2]      0's
neighbours
#      1   2              adj[1] = [0]         1 only
touches 0
#         / \             adj[2] = [0, 3, 4]   2 touches 0,
3 and 4
#        3   4            adj[3] = [2]
#                         adj[4] = [2]

# input: n, then n-1 lines "u v" (1-indexed), e.g. "1 3" is
the edge (0, 2)
n = int(nxt())
adj = [[] for _ in range(n)]       # n empty lists, one per
node
for _ in range(n - 1):             # a tree always has
exactly n-1 edges
    u = int(nxt()) - 1             # -1: input counts from
1, we count from 0
    v = int(nxt()) - 1
    adj[u].append(v)               # record "v is a
neighbour of u" ...
    adj[v].append(u)               # ... and "u is a
neighbour of v"
```

Navigate it: one BFS from the root. This single loop is the workhorse for almost every tree problem. It visits every node once, moving outward from the root level by level, and fills in the three arrays you keep needing: who my parent is, how deep I am, and the order nodes were discovered in. `seen` stops us from walking back up the edge we came from (in a tree, the only already-visited neighbour is the parent).

```
root = 0
parent = [-1] * n          # parent[v] = v's neighbour
toward the root
depth = [0] * n            # depth[v]  = number of edges
from the root to v
order = []                 # nodes in the order BFS reached
them (top-down)
seen = [False] * n
seen[root] = True
q = deque([root])
while q:
    u = q.popleft()        # take the oldest node in the
queue
    order.append(u)
    for v in adj[u]:
        if not seen[v]:    # every unseen neighbour is a
child of u
            seen[v] = True
            parent[v] = u
            depth[v] = depth[u] + 1
            q.append(v)    # it will be processed after this
level

# on the example tree, after this loop:
#   parent = [-1, 0, 0, 2, 2]   1 and 2 hang off 0; 3 and 4
hang off 2
#   depth  = [ 0, 1, 1, 2, 2]   root is 0 edges deep, 3 and
4 are 2 deep
#   order  = [ 0, 1, 2, 3, 4]   level by level: root, its
children, theirs
```

After this loop: `depth[v]` is the distance from the root to v (each tree edge costs 1), the children of u are `[v for v in adj[u] if parent[v] == u]`, and a leaf is a non-root node with `len(adj[v]) == 1`.

Walk a path back up. `parent` is a chain of links pointing toward the root — follow it to recover the actual path from any node to the root:

```
path = []
v = target
while v != -1:             # the root's parent is -1, so the
loop stops there
    path.append(v)
    v = parent[v]
path.reverse()             # now path = [root, ..., target]
```

Process the tree bottom-up (tree DP). `order` is top-down (parents before children), so `reversed(order)` is bottom-up (children before parents). Sweep it to combine answers of children into their parent — no recursion needed. Subtree size is the simplest example:

```
size = [1] * n             # every node counts itself
for u in reversed(order):  # children are done before their
parent
    if parent[u] != -1:
        size[parent[u]] += size[u]
# example tree: size = [5, 1, 3, 1, 1] — node 2's subtree is
{2, 3, 4}
```

The same two lines solve any "compute something for every subtree" problem; only the combine step changes. For example, the sum of depths in each subtree:

```
dp = list(depth)                   # dp[u] starts as u's own
value
for u in reversed(order):
    if parent[u] != -1:
        dp[parent[u]] += dp[u]     # fold u's subtree into
its parent
```

Distance between two nodes = `depth[a] + depth[b] - 2 * depth[lca(a, b)]` (see LCA below). For a handful of queries, a plain BFS from a (the loop above, reading `depth[b]`) is simpler.

Diameter (longest path in the tree) — BFS from any node and note the farthest node a; BFS again from a and note the farthest node b. dist(a, b) is the diameter. (Why it works: the farthest node from anywhere is always an endpoint of some diameter.)

LCA — the *lowest common ancestor* of a and b: where the paths root→a and root→b part ways. It gives distances (formula below) and "does the path a–b pass through c". Binary lifting makes queries fast by precomputing jump tables: `up[k][v]` = the ancestor 2^k steps above v, built like doubling — a 4-jump is a 2-jump of a 2-jump. Any upward distance is then a sum of power-of-two jumps (its binary digits). O(n log n) build, O(log n) per query.

```
LOG = max(1, (n - 1).bit_length())
up = [[0] * n for _ in range(LOG)]      # up[k][v] = 2^k-th
ancestor of v
for v in range(n):
    up[0][v] = parent[v] if parent[v] != -1 else root
for k in range(1, LOG):
    for v in range(n):
        up[k][v] = up[k - 1][up[k - 1][v]]

def lca(a, b):
    if depth[a] < depth[b]:
        a, b = b, a
    diff = depth[a] - depth[b]
    for k in range(LOG):                # lift a up to the
depth of b
        if diff >> k & 1:
            a = up[k][a]
    if a == b:
        return a
    for k in range(LOG - 1, -1, -1):    # lift both while
they stay different
        if up[k][a] != up[k][b]:
            a = up[k][a]
            b = up[k][b]
    return up[0][a]

def tree_dist(a, b):
    return depth[a] + depth[b] - 2 * depth[lca(a, b)]
# k-th ancestor of a: same bit loop as 'diff'
```

Euler tour — makes every subtree a contiguous range; ancestor checks in O(1).

```
tin = [0] * n                  # tin[u]  = the "time" DFS
first entered u
tout = [0] * n                 # tout[u] = the time DFS
finally left u
timer = 0
stack = [(root, 0)]            # (node, how many of its
neighbours tried so far)
while stack:
    u, i = stack[-1]           # peek at the node we're
currently inside
    if i == 0:
        tin[u] = timer         # first time here: stamp the
entry time
        timer += 1
    if i < len(adj[u]):        # still has neighbours to try
        stack[-1] = (u, i + 1)
        v = adj[u][i]
        if v != parent[u]:     # don't walk back up the tree
            stack.append((v, 0))     # descend into child v
    else:                      # all children fully explored:
        tout[u] = timer        # stamp the exit time and
        stack.pop()            # return to the parent
# u is an ancestor of v  <=>  tin[u] <= tin[v] and tout[v]
<= tout[u]
# subtree of u = positions [tin[u], tout[u]) ->
Fenwick/segment tree over tin order
```

Linked list: insert/remove at a specific spot in O(1). Python has no built-in linked list (`list.insert(i, x)` and `list.pop(i)` shift everything — O(n); `deque` is fast only at the two ends). When a problem removes/inserts elements in the middle many times (Josephus-style eliminations, "delete me and my neighbours join up"), build the linked list out of two arrays instead of node objects: `nxt[i]` and `prv[i]` hold the INDEX of the element after/before element i. Rewiring a few indices is the whole operation.

```
# start: elements 0..n-1 in their input order, linked in a
row
nxt = list(range(1, n + 1))        # nxt[i] = i+1 ...
nxt[n - 1] = -1                    # ... except the last,
which has no 'after'
prv = list(range(-1, n - 1))       # prv[i] = i-1; the first
has no 'before'

def remove(i):                     # unlink element i - its
neighbours join up
    if prv[i] != -1:
        nxt[prv[i]] = nxt[i]       # the one before i now
points past i
    if nxt[i] != -1:
        prv[nxt[i]] = prv[i]       # the one after i now
points back past i

def insert_after(i, j):            # link element j in right
after element i
    nxt[j] = nxt[i]                # j points at what used
to follow i
    prv[j] = i                     # j points back at i
    if nxt[i] != -1:
        prv[nxt[i]] = j            # i's old follower points
back at j
    nxt[i] = j                     # i now points at j

# walk the list: i = head;  while i != -1: visit(i); i =
nxt[i]
# removed elements stay in the arrays but nothing points at
them any more
# circular version (Josephus): nxt[n-1] = 0, prv[0] = n-1,
drop the -1 checks
```

10. Range data structures

When you need this. The giveaway is an array plus MANY queries that *mix two kinds of operations*: "change one value" and "ask about a range" (sum / min / max of `a[l..r]`). Answering one range query by scanning is O(n), so with n and q both around 10^5 the naive way is 10^{10} steps — hopeless. The structures below make BOTH operations O(log n), so the whole thing runs in about n + q log n. Typical disguises: "after each update, print the sum of a segment"; "how many soldiers between positions l and r" with soldiers appearing/leaving; counting inversions; "k-th smallest currently alive value"; subtree queries after an Euler tour (section 9). If the array NEVER changes, you don't need any of this — prefix sums (section 5) answer sums in O(1), and a sparse table (below) answers min/max in O(1).

Which one to pick:

updates	queries	use
none	range sum	prefix sums (section 5)
none	range min/max/gcd	sparse table
single cell changes	range sum	Fenwick — shortest code
single cell changes	range min/max/any op	segment tree
whole-range updates (`a[l..r] += v`)	range queries	lazy segment tree
whole-range updates	single cell reads	Fenwick over a difference array (noted below)

Fenwick tree (BIT) — point update + prefix sum, O(log n) each. The idea: cell i of the tree stores the sum of a block of the array whose LENGTH is i's lowest set bit (cell 12 = 0b1100 covers 4 elements). Any prefix 1..i splits into a few such blocks, and `i & -i` (the lowest set bit, section 16) is how the code hops from block to block — down for queries, up for updates. All of it is these two tiny loops. 1-indexed.

```
tree = [0] * (n + 1)

def add(i, delta):                 # a[i] += delta, i in 1..n
    while i <= n:
        tree[i] += delta
        i += i & -i                # i & -i = lowest set bit

def pref(i):                       # a[1] + ... + a[i]
    s = 0
    while i > 0:
        s += tree[i]
        i -= i & -i
    return s

def range_sum(l, r):
    return pref(r) - pref(l - 1)
# range UPDATE + point query: put the DIFFERENCE array in
the Fenwick:
#   update(l, v) and update(r+1, -v);  value at i = pref(i)
```

Counting inversions: coordinate-compress, sweep right to left; for each element `answer += pref(rank - 1)` then `add(rank, 1)`.

Segment tree (iterative) — point update + range query for any associative op (min, max, sum, gcd…). Picture a tournament bracket over the array: leaves are the values, each internal node holds OP of its two children, the root covers everything. An update replays one leaf's matches up to the root; a query stitches together O(log n) already-computed nodes that exactly tile [l, r). Stored flat in one list: node i's children are 2i and 2i+1, leaves live at n..2n-1.

```
OP = min                           # or max, or (lambda a, b: a
+ b), or math.gcd
IDENTITY = INF                     # value that never wins: 0
for sum, -INF for max

tree = [IDENTITY] * (2 * n)
tree[n:2 * n] = a                  # leaves live at n..2n-1
for i in range(n - 1, 0, -1):  # fill parents from the
leaves up
    tree[i] = OP(tree[2 * i], tree[2 * i + 1])

def update(i, value):              # a[i] = value
    i += n                         # jump to i's leaf
    tree[i] = value
    i //= 2                        # i's parent
    while i >= 1:                  # replay the "matches" up to
the root
        tree[i] = OP(tree[2 * i], tree[2 * i + 1])
        i //= 2

def query(l, r):                   # OP over a[l..r-1]  (half-
open!)
    res = IDENTITY
    l += n                         # walk both boundaries up
from their leaves,
    r += n                         # grabbing nodes that lie
fully inside [l, r)
    while l < r:
        if l % 2 == 1:             # l is a right child: take it
and move right
            res = OP(res, tree[l])
            l += 1
```

```
        if r % 2 == 1:          # r is a right child: step
left and take it
            r -= 1
            res = OP(res, tree[r])
        l //= 2
        r //= 2
    return res
```

Lazy segment tree — range add + range min. Recursion depth is only log n, so recursion is safe here.

```
class LazySegTree:
    def __init__(self, a):
        self.n = len(a)
        self.mn = [0] * (4 * self.n)     # min of each
node's range
        self.lz = [0] * (4 * self.n)     # pending "+= lz"
for the whole range
        self._build(1, 0, self.n - 1, a)

    def _build(self, x, l, r, a):
        if l == r:
            self.mn[x] = a[l]
            return
        m = (l + r) // 2
        self._build(2 * x, l, m, a)
        self._build(2 * x + 1, m + 1, r, a)
        self.mn[x] = min(self.mn[2 * x], self.mn[2 * x + 1])

    def _push(self, x):                  # hand the pending
add down to the children
        if self.lz[x] != 0:
            for c in (2 * x, 2 * x + 1):
                self.mn[c] += self.lz[x]
                self.lz[c] += self.lz[x]
            self.lz[x] = 0

    def add(self, ql, qr, v, x=1, l=0, r=None):      #
a[ql..qr] += v
        if r is None:
            r = self.n - 1
        if qr < l or r < ql:
            return
        if ql <= l and r <= qr:
            self.mn[x] += v
            self.lz[x] += v
            return
        self._push(x)
        m = (l + r) // 2
        self.add(ql, qr, v, 2 * x, l, m)
        self.add(ql, qr, v, 2 * x + 1, m + 1, r)
        self.mn[x] = min(self.mn[2 * x], self.mn[2 * x + 1])

    def query_min(self, ql, qr, x=1, l=0, r=None):   # min
over a[ql..qr]
        if r is None:
            r = self.n - 1
        if qr < l or r < ql:
            return INF
        if ql <= l and r <= qr:
            return self.mn[x]
        self._push(x)
        m = (l + r) // 2
        return min(self.query_min(ql, qr, 2 * x, l, m),
                   self.query_min(ql, qr, 2 * x + 1, m + 1,
r))
# range-sum version: also store each node's range length;
apply lz * length to sums
```

Sparse table — range min/max/gcd on a STATIC array; O(1) query after O(n log n) build.

```
sp = [list(a)]                       # sp[k][i] = min of a[i
.. i + 2^k - 1]
k = 1
while (1 << k) <= n:
    prev = sp[-1]
    half = 1 << (k - 1)
    sp.append([min(prev[i], prev[i + half]) for i in range(n
- (1 << k) + 1)])
    k += 1

def query_min(l, r):                 # min over a[l..r]
inclusive
    k = (r - l + 1).bit_length() - 1
    return min(sp[k][l], sp[k][r - (1 << k) + 1])   # two
overlapping halves
```

No sorted multiset in Python. Substitutes: heapq + lazy deletion (keep a Counter of deleted values; when popping, discard tops that are marked deleted); `bisect.insort` on a list (O(n) insert — fine up to ~10^5 operations); a Fenwick tree over compressed values (gives count-less-than, k-th smallest); two heaps for a running median.

11. Dynamic programming

How to spot a DP problem. The strongest signals:

- The question asks for a **count** ("how many ways…"), an **optimum** ("minimum cost / maximum value…"), or **possibility** ("can you reach exactly…") — and a greedy rule is easy to break with a counterexample.
- Choices now affect what's allowed later (take an item → less capacity left), but the future only depends on a small **summary** of the past (how much capacity is left), not on the full history. That summary is your state.
- Constraints whisper the shape: $n \le 20$ → bitmask DP over subsets; $n \le 500$ → $O(n^2)$/$O(n^3)$ table; "answer modulo 10^9+7" → almost always a counting DP; two strings → 2D table over prefixes; "digits of N" → digit DP.
- Classic phrasings: subsequences, partitions, ways to climb/tile/pay, "pick non-adjacent", turn-based games.

What DP actually is. Brute force explores every sequence of choices — a tree that explodes exponentially. DP notices that millions of those branches end up in the SAME situation ("7 capacity left", "at position i with 3 coins used") and that identical situations have identical futures. So instead of exploring branches, you fill a table with one entry per *situation*, each computed once. DP = brute force + never solving the same situation twice.

Deriving one from scratch (think this way every time): *fewest coins from {1, 3, 4} to pay 6.* Greedy fails (4+1+1 = 3 coins, but 3+3 = 2). Instead, interrogate the LAST decision: the final coin was 1, 3 or 4 — whichever it was, what came before is the *same problem with a smaller total*. So define `dp[t]` = fewest coins to pay t, and `dp[6] = 1 + min(dp[5], dp[3], dp[2])`. Base: `dp[0] = 0`. Fill t = 1..6 in order and read `dp[6]`. Every DP below is this move — "condition on the last decision" — applied to a different notion of 'situation'.

Recipe. 1) State in words: "dp[i] = best … considering the first i items" — if you can't say it in one sentence, the state is wrong. 2) Transition: how dp[state] follows from smaller states (= the last decision made). 3) Order: everything a state needs is computed before it. 4) Base case; unreachable states ±INF. 5) Where does the answer sit? Sanity-check: #states × work per state must fit the time limit. In PyPy write iterative tables, not recursion.

Map of the patterns below — which box solves what:

- **Knapsack family**: pick items under a budget (weight/money/time). State = budget used.
- **Counting tables**: the same tables with `+` instead of `min`/`max` — "how many ways".
- **LIS / LCS / edit distance**: the state is a *prefix* (or a pair of prefixes) of a sequence.
- **Kadane**: best contiguous run; state = best run ending exactly here.
- **Bitmask / TSP**: the order of visiting matters and $n \le$ ~17; state = (set visited, current position).
- **Interval DP**: cheapest way to merge/build a whole segment; state = a sub-interval [l, r].
- **Digit DP**: count numbers ≤ N with a property; state = (digit position, still-tight?, what you track).
- **DP on a DAG / tree DP**: states are graph nodes; order = toposort (section 8) or bottom-up (section 9).

```
# 0/1 knapsack: n items, each used AT MOST ONCE, capacity C,
maximise value.
# dp[c] = best value achievable with capacity exactly <= c,
using items seen so far
dp = [0] * (C + 1)
for w, v in items:                   # consider items one by
one
    for c in range(C, w - 1, -1):  # capacity DOWNWARD: dp[c
- w] is then still
        #                             "before this item", so
the item can't be
        #                             counted twice in one
pass
        dp[c] = max(dp[c],           # don't take the item ...
                    dp[c - w] + v) # ... or take it: value
v, uses w capacity
```

```
# answer: dp[C]

# unbounded knapsack: UNLIMITED copies of each item -> capacity loop goes UP,
# so dp[c - w] may already include this item, which is exactly what we want
for w, v in items:
    for c in range(w, C + 1):
        dp[c] = max(dp[c], dp[c - w] + v)

# minimum number of coins to pay t: same table, min instead of max
# dp[0] = 0, rest INF;  dp[t] = min(dp[t], dp[t - coin] + 1) for every coin

# count ways to reach total t when ORDER MATTERS (dice rolls: 1+2 != 2+1):
# one loop over totals, inner loop over faces
#   dp[0] = 1;  dp[t] = sum(dp[t - f] for every face f)

# count ways when order does NOT matter (coin combos: 1+2 == 2+1):
# OUTER loop over coins, INNER loop over totals — processing coins one at a
# time imposes a fixed order, so each combination is counted exactly once
ways = [0] * (T + 1)
ways[0] = 1                          # one way to pay 0: use nothing
for coin in coins:
    for t in range(coin, T + 1):
        ways[t] += ways[t - coin]  # ways using this coin at least once more
```

```
# which-sums-are-reachable as a bitmask over a big int (very fast trick):
bits = 1                             # bit s set <=> sum s is reachable
for w in weights:
    bits |= bits << w
reachable = bits >> target & 1

# reconstructing chosen items (knapsack): keep a 2D table dp[i][c] (first i items),
# then walk backwards from dp[n][C]: if dp[i][c] != dp[i-1][c], item i was taken.
```

```
# Longest increasing subsequence in O(n log n).
# tails[k] = the SMALLEST value that can end an increasing subsequence of
# length k+1. tails stays sorted, so each element binary-searches its spot:
# it either extends the longest subsequence so far, or improves (lowers) the
# ending value of some shorter one.
tails = []
pos = []                             # pos[i] = LIS-length-1 achieved at element i
for x in a:
    p = bisect.bisect_left(tails, x)     # first ending value >= x
    pos.append(p)                        # (bisect_right for non-DEcreasing LIS)
    if p == len(tails):
        tails.append(x)              # x extends the best subsequence by one
    else:
        tails[p] = x                 # x is a cheaper ending for length p+1
L = len(tails)                       # the LIS length (tails is NOT the subsequence!)

# reconstruct one LIS: scan right-to-left collecting pos == L-1, L-2, ...
indices = []
need = L - 1
for i in range(n - 1, -1, -1):
    if pos[i] == need:
        indices.append(i)
        need -= 1
indices.reverse()
```

```
# Longest common subsequence of s and t, O(|s|*|t|).
# dp[i][j] = LCS length of the first i chars of s and the first j chars of t.
# Row/column 0 stay 0 (an empty prefix shares nothing) — that's the base case.
dp = [[0] * (len(t) + 1) for _ in range(len(s) + 1)]
for i in range(1, len(s) + 1):
    for j in range(1, len(t) + 1):
        if s[i - 1] == t[j - 1]:             # last chars match:
            dp[i][j] = dp[i - 1][j - 1] + 1    #   pair them up, +1
        else:                                # they can't both be in the LCS:
            dp[i][j] = max(dp[i - 1][j],       #   drop s's last char,
                           dp[i][j - 1])       #   or drop t's — keep the better
# reconstruct: from (n, m) step to whichever cell produced the value; diagonal = keep char

# edit distance (insert/delete/replace, all cost 1): dp[i][0] = i, dp[0][j] = j
#   same shape; if chars equal dp[i-1][j-1]
#   else 1 + min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])
```

```
# Kadane: maximum sum of a (non-empty) subarray
best = cur = a[0]
for x in a[1:]:
    cur = max(x, cur + x)          # extend the run, or restart at x
    best = max(best, cur)
```

```
# Bitmask DP / TSP (visit every city once, min total distance; n <= ~17).
# The state must remember WHICH cities are done (a set -> a bitmask, section 16)
# and where we currently stand:
# dp[mask][u] = cheapest way to have visited exactly the set 'mask', ending at u
dp = [[INF] * n for _ in range(1 << n)]
dp[1][0] = 0                         # mask 1 = {0}: start at city 0, cost 0
for mask in range(1 << n):         # increasing mask = valid order: a set is
    for u in range(n):             # always a bigger number than its subsets
        if dp[mask][u] == INF or not (mask >> u) & 1:
            continue               # unreachable, or u is not even in the set
        for v in range(n):         # try travelling on to each unvisited v
            if (mask >> v) & 1:
                continue           # v already visited
            new_mask = mask | (1 << v)          # the set with v added
            cost = dp[mask][u] + dist[u][v]     # dist = matrix of pair distances
            if cost < dp[new_mask][v]:
                dp[new_mask][v] = cost
full = (1 << n) - 1                # all n cities visited
ans = min(dp[full][u] + dist[u][0] for u in range(n))   # return to start = cycle;
#
drop "+ dist[u][0]" for a path
```

```
# interval DP (matrix chain / merging costs): solve [l, r] from its split points.
# The LENGTH loop is the outer loop so smaller intervals are ready first.
for length in range(2, n + 1):
    for l in range(n - length + 1):
        r = l + length - 1
        dp[l][r] = min(dp[l][k] + dp[k + 1][r] + cost(l, k, r) for k in range(l, r))
```

```
# digit DP: count x in [0, N] with a digit property (depth <= 19, recursion is fine)
from functools import lru_cache
def count_up_to(N):
    s = str(N)
    @lru_cache(maxsize=None)
    def go(i, tight, state):
        # i = position; tight = "still equal to N's prefix"; state = whatever you track
        if i == len(s):
            return 1 if good(state) else 0
        limit = int(s[i]) if tight else 9
        total = 0
        for digit in range(0, limit + 1):
            total += go(i + 1, tight and digit == limit, step(state, digit))
        return total
    result = go(0, True, initial_state)
    go.cache_clear()
    return result
# answer for [L, R] = count_up_to(R) - count_up_to(L - 1); mind leading zeros
```

Game theory / Grundy. A position is losing iff every move leads to a winning position: `win[x] = any(not win[y] for y in moves(x))`. Several independent games: XOR their Grundy numbers, 0 = losing. `g[x] = mex{g[y] : y in moves(x)}` where mex = smallest missing non-negative integer. Nim: XOR of pile sizes.

12. Strings

Spot it. "Find/count occurrences of a pattern in a text" → KMP or Z (naive find is O(n·m) — too slow when both are big). "Are these two substrings equal", many times → hashing. Palindromes everywhere → Manacher. A dictionary of words queried by prefix → trie. Smallest repeating block / period → KMP's prefix function.

KMP — find a pattern in a text without ever re-reading text characters. Its engine is the *prefix function*: `pi[i]` = the length of the longest proper prefix of `s[:i+1]` that is also its suffix (for "abcab", pi ends at 2: "ab" is both). Why that helps: after a mismatch you don't restart the pattern from scratch — pi tells you the longest partial match that MUST still hold, so you resume from there. Total O(n + m). (Honest shortcut: Python's built-in `text.find(pattern)` is fast C and usually enough — reach for KMP when you need ALL occurrences or the pi array itself, e.g. periods.)

```
def prefix_function(s):
    # pi[i] = length of the longest proper prefix of s[:i+1]
that is also its suffix
    pi = [0] * len(s)
    for i in range(1, len(s)):
        j = pi[i - 1]
        while j > 0 and s[i] != s[j]:
            j = pi[j - 1]          # current match broke:
fall back to the next
            #                        shorter prefix that
still matches
        if s[i] == s[j]:
            j += 1                 # extend the match by
this character
        pi[i] = j
    return pi

def find_all(pattern, text):       # all start indices of
pattern inside text
    m = len(pattern)
    # glue them with a separator that occurs in neither;
wherever pi reaches
    # the full pattern length, the pattern just ended in the
text
    pi = prefix_function(pattern + "\x00" + text)
    return [i - 2 * m for i in range(len(pi)) if pi[i] == m]
# smallest period of s: n - pi[n-1], if that divides n
```

Z-function — `z[i]` = how many characters, starting at position i, match the START of the string (for "aabxaab": z[4] = 3, since "aab…" reappears there). Same uses as KMP, often easier to reason about: to search, build z of `pattern + '#' + text` and every z equal to len(pattern) is a hit. The code stays O(n) by remembering the rightmost match window and reusing earlier z-values inside it instead of re-comparing.

```
def z_function(s):
    n = len(s)
    z = [0] * n
    if n > 0:
        z[0] = n
    l = r = 0                      # [l, r) = rightmost
window known to match the prefix
    for i in range(1, n):
        if i < r:                  # i sits inside a known
match -> its start
            z[i] = min(r - i, z[i - l])     # mirrors
position i-l; reuse that
            #                        answer (capped at the
window edge)
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1              # extend by actual
comparison past the known part
        if i + z[i] > r:
            l, r = i, i + z[i]     # this match reaches
further right: remember it
    return z
# matching: z of (pattern + '#' + text); occurrences where
z[i] == len(pattern)
```

Polynomial hashing — treat a string as a number in a huge base (like "425" = 4·100 + 2·10 + 5, but with characters as digits), kept modulo a big prime. Precompute the hash of every prefix once; then the hash of ANY substring is one subtraction — so "are these two substrings equal?" becomes comparing two numbers, O(1). Equal hashes *could* collide, but with a random base the chance is negligible. This is the flexible duct tape of string problems: equality, palindrome checks, comparing across different strings.

```
import random
MOD = (1 << 61) - 1
BASE = random.randrange(256, MOD)   # random base resists
constructed anti-hash tests

def build_hash(s):
    h = [0] * (len(s) + 1)          # h[i] = hash of s[:i]
    pw = [1] * (len(s) + 1)         # pw[i] = BASE^i
    for i, ch in enumerate(s):
        h[i + 1] = (h[i] * BASE + ord(ch)) % MOD
        pw[i + 1] = pw[i] * BASE % MOD
    return h, pw

def get_hash(h, pw, l, r):          # hash of s[l:r], half-
open
    return (h[r] - h[l] * pw[r - l]) % MOD
# palindrome checks: compare with hashes of the reversed
string (flip indices)
# if paranoid: use two different (BASE, MOD) pairs and
compare both
```

Trie — a tree of characters: all words sharing a prefix share a path from the root, so "how many words start with 'pre'" or "is this a prefix of a dictionary word" is just walking the path. In Python, a dict of dicts (XOR problems use bits as the characters):

```
root = {}
for word in words:
    node = root
    for ch in word:
        node = node.setdefault(ch, {})
    node['$'] = node.get('$', 0) + 1    # end-of-word marker
/ counter
```

Manacher — for every position, the length of the longest palindrome centred there, all in O(n). (Checking each centre naively is O(n²).) The trick: inside a known big palindrome, the left half mirrors the right, so each new centre starts from its mirror's answer instead of from zero. The # separators make even-length palindromes have a centre too.

```
def manacher(s):
    t = "#" + "#".join(s) + "#"     # insert separators so
all palindromes are odd
    n = len(t)
    p = [0] * n                     # p[i] = radius; also =
palindrome length in s
    centre = right = 0
    for i in range(n):
        if i < right:               # i lies inside a known
palindrome:
            p[i] = min(right - i,   # start from the answer
of i's MIRROR
                       p[2 * centre - i])    # position,
capped at the edge
        while i - p[i] - 1 >= 0 and i + p[i] + 1 < n and t[i
- p[i] - 1] == t[i + p[i] + 1]:
            p[i] += 1               # grow outward by real
comparisons
        if i + p[i] > right:        # new palindrome reaches
further right:
            centre, right = i, i + p[i]      # it becomes
the known one
    return p
# longest palindromic substring = max(p); its start in s =
(i - p[i]) // 2
```

Parsing tips. Nested structures → a stack (push on open, resolve on close). Expression evaluation → recursive descent on tokens. The `re` module is available: `re.findall(r"-?\d+", line)` grabs every integer on a line.

13. Number theory and counting

Spot it. "Answer modulo 10^9+7" (→ modular arithmetic + factorials/binomials), huge exponents (→ `pow(a, b, m)`), divisors/factors/primality of numbers up to 10^{18} (→ Miller–Rabin/Pollard), "for every number up to n" (→ a sieve), remainders with several moduli (→ CRT), and a linear recurrence (like Fibonacci) asked at $n \approx 10^{18}$ (→ matrix power).

The one idea behind "mod 10^9+7". Counting answers get astronomically large, so the judge asks only for the remainder after dividing by a big prime. The useful fact: you may take the remainder after EVERY step — `(a + b) % M`, `(a * b) % M` — and the final remainder is the same as computing exactly and reducing once. So just sprinkle `% MOD` on every `+`, `-`, `*`. The exception is division: `(a / b) % M` is meaningless. Instead multiply by b's *modular inverse* —

the number b^{-1} with `b * b⁻¹ ≡ 1 (mod M)` — which exists exactly because M is prime (that's why judges pick a prime!). `pow(b, -1, MOD)` computes it.

```
math.gcd(a, b)
math.lcm(a, b)
pow(a, b, m)                          # a^b mod m
pow(a, -1, m)                         # inverse of a mod m
(gcd(a, m) must be 1)
```

Extended gcd and CRT. Plain gcd says *how much* a and b share; extended gcd also hands you a certificate — x and y with a·x + b·y = gcd. That certificate is what lets you solve equations in remainders: "find x with a·x ≡ c (mod m)", and the CRT below, which stitches two separate remainder conditions ("x ≡ 2 mod 5 AND x ≡ 3 mod 7") into a single one ("x ≡ 17 mod 35"). Use CRT when a problem tracks something modulo several different bases.

```
def extgcd(a, b):                     # returns (g, x, y) with
a*x + b*y = g = gcd(a, b)
    if b == 0:
        return a, 1, 0                # gcd(a, 0) = a, and a*1
+ 0*0 = a
    g, x, y = extgcd(b, a % b)        # solve the smaller
problem first ...
    return g, y, x - (a // b) * y     # ... then rewrite its
answer in terms of a, b
    # (recursion depth = number of gcd steps, ~log — safe)
# solve a*x ≡ c (mod m): solvable iff gcd(a, m) divides c

def crt(r1, m1, r2, m2):
    # smallest x >= 0 with x ≡ r1 (mod m1) AND x ≡ r2 (mod
m2); None if impossible
    g, p, q = extgcd(m1, m2)          # p is the key: m1*p ≡ g
(mod m2)
    if (r2 - r1) % g != 0:
        return None                   # the two conditions
contradict each other
    lcm = m1 // g * m2                # the combined condition
repeats every lcm
    # start from r1 (satisfies condition 1) and add the
right multiple of m1
    # (keeps condition 1 true) to also satisfy condition 2:
    x = (r1 + (r2 - r1) // g * p % (m2 // g) * m1) % lcm
    return x, lcm                     # all solutions: x,
x+lcm, x+2*lcm, ...
```

Sieve, factorisation, divisors. Two regimes, don't mix them up. Questions about MANY numbers up to n ("which of 1..10^6 are prime", "smallest factor of each") → sieve: cross out multiples of each prime once, O(n log log n) total, far cheaper than testing each number alone. Questions about ONE number → trial division up to √x is enough while x ≤ ~10^{14} (a factor pair always has one member ≤ √x — that's why the loops stop there).

```
def sieve(n):
    # is_prime[x] answers "is x prime?" for every x up to n
at once.
    # Start assuming everything is prime, then cross out
what can't be.
    is_prime = [True] * (n + 1)
    is_prime[0] = is_prime[1] = False
    for i in range(2, math.isqrt(n) + 1):
        if is_prime[i]:               # i survived -> it is a
real prime
            for j in range(i * i, n + 1, i):
                is_prime[j] = False # cross out every
multiple of i.
                # start at i*i: smaller multiples like 3*i
had a smaller
                # prime factor and were already crossed out
    return is_prime
# list of the primes themselves: [x for x in range(n + 1) if
is_prime[x]]

def factorize(x):                     # trial division, O(sqrt
x) — fine up to ~10^14
    factors = []
    d = 2
    while d * d <= x:
        while x % d == 0:
            factors.append(d)
            x //= d
        d += 1
    if x > 1:
        factors.append(x)
    return factors

def divisors(x):
    result = []
    d = 1
    while d * d <= x:
        if x % d == 0:
            result.append(d)
            if d != x // d:
                result.append(x // d)
        d += 1
    return result
# many factorisation queries up to n: sieve of smallest
prime factors (spf),
#   spf[j] = smallest prime dividing j; divide by spf
repeatedly -> O(log x) each
```

Miller–Rabin and Pollard's rho — for ONE number around 10^{18}, where √x trial division (10^9 steps) is too slow. Miller–Rabin answers "is n prime?" fast: a prime must pass a certain squaring pattern for every base a, and checking the fixed dozen bases below is PROVEN to be exact for all n < $3 \cdot 10^{24}$ — no randomness in practice. Pollard's rho finds *some* factor of a composite n: it runs a pseudo-random walk whose values repeat modulo an unknown factor far sooner than modulo n (birthday paradox), and a gcd catches that early repeat. Recurse on both halves for the full factorisation.

```
def is_prime(n):
    if n < 2:
        return False
    for p in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
        if n % p == 0:                # quick exit on small
factors
            return n == p
    d = n - 1
    s = 0
    while d % 2 == 0:                 # write n-1 as d * 2^s
with d odd
        d //= 2
        s += 1
    for a in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
# the witnesses
        # if n is prime, the sequence a^d, a^(2d), a^(4d),
... (mod n)
        # must hit n-1 before it hits 1 (or start at 1).
Composites fail
        # this for at least one of the witnesses above.
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue                  # this witness is
satisfied
        for _ in range(s - 1):
            x = x * x % n             # keep squaring, looking
for n-1
            if x == n - 1:
                break                 # satisfied after all
        else:                         # NOTE: for-ELSE runs
only if no break
            return False              # never reached n-1 ->
definitely composite
    return True

import random
def pollard(n):                       # one non-trivial factor
of a COMPOSITE n
    if n % 2 == 0:
        return 2
    while True:
        x = random.randrange(2, n)
        y = x
        c = random.randrange(1, n)
        d = 1
        while d == 1:
            x = (x * x + c) % n
            y = (y * y + c) % n
            y = (y * y + c) % n       # y moves twice as fast
            d = math.gcd(abs(x - y), n)
        if d != n:
            return d

def full_factor(n):                   # works for n up to
~10^18
    if n == 1:
        return []
    if is_prime(n):
        return [n]
    d = pollard(n)
    return full_factor(d) + full_factor(n // d)
```

Binomials mod a prime. Most counting answers decompose into C(n, k) = "ways to choose k things out of n" (paths in a grid, teams, arrangements with ties). C(n,k) = n! / (k!(n–k)!) has divisions, so mod a prime you multiply by inverse factorials instead. Precompute all

factorials and inverse factorials ONCE (the loops below), then every C(n, k) in the problem costs three multiplications.

```
MOD = 10**9 + 7
MAXN = 2 * 10**5
fact = [1] * (MAXN + 1)
for i in range(1, MAXN + 1):
    fact[i] = fact[i - 1] * i % MOD
inv_fact = [1] * (MAXN + 1)
inv_fact[MAXN] = pow(fact[MAXN], MOD - 2, MOD)
for i in range(MAXN, 0, -1):
    inv_fact[i - 1] = inv_fact[i] * i % MOD

def C(n, k):
    if k < 0 or k > n:
        return 0
    return fact[n] * inv_fact[k] % MOD * inv_fact[n - k] %
MOD
# small/exact cases: math.comb(n, k)
```

Matrix power — for a sequence where each term is a fixed mix of the previous k terms (Fibonacci: F(n) = F(n−1) + F(n−2)) asked at an absurd index like n = 10^{18}. One step of the recurrence is "multiply the state vector by a fixed matrix", so n steps = that matrix to the n-th power — and repeated squaring (M, M^2, M^4, …) reaches the n-th power in $\log_2 n \approx 60$ multiplications instead of 10^{18}.

```
def mat_mul(A, B, mod):
    rows, mid, cols = len(A), len(B), len(B[0])
    C = [[0] * cols for _ in range(rows)]
    for i in range(rows):
        for j in range(cols):
            s = 0
            for t in range(mid):
                s += A[i][t] * B[t][j]
            C[i][j] = s % mod
    return C

def mat_pow(A, e, mod):
    size = len(A)
    # R starts as the identity matrix (the "1" of matrix
multiplication)
    R = [[1 if i == j else 0 for j in range(size)] for i in
range(size)]
    while e > 0:                    # walk the bits of the
exponent:
        if e & 1:                   # this bit is set ->
multiply in the
            R = mat_mul(R, A, mod)  # current power of A
        A = mat_mul(A, A, mod)      # A, A^2, A^4, A^8, ...
by squaring
        e >>= 1
    return R                        # e.g. A^13 = A^8 * A^4
* A^1 (13 = 1101)
# Fibonacci: M = [[1, 1], [1, 0]];  M^n gives F(n) at [0][1]
```

Counting facts. Sum 1..n = n(n+1)/2; sum of squares = n(n+1)(2n+1)/6. Stars and bars: ways to write n as k non-negative parts = C(n+k−1, k−1). Inclusion–exclusion: $|A \cup B \cup C| = \Sigma|\text{single}| - \Sigma|\text{pairs}| + |\text{triple}|$. Catalan number C(2n, n)/(n+1): balanced brackets, binary trees, non-crossing paths. Expected value is linear even for dependent events. Pigeonhole: n+1 items in n boxes → some box has 2. Handshake: Σ degrees = 2·edges.

Exact arithmetic. Python ints are arbitrary precision — never overflow. Avoid floats when possible: compare a/b vs c/d as a·d vs c·b; `from fractions import Fraction` is exact (but slow in bulk). Base conversion: `int(s, b)` parses; to print in base b, repeatedly divmod and reverse the digits.

14. Geometry

Spot it. Points, segments or polygons in the input. Use integers for as long as possible; convert to float only at the end (floats lie about equality; integers never do).

The one tool: the cross product. `cross(o, a, b)` answers a single question: standing at o and facing a, is b to your LEFT (result > 0), to your RIGHT (< 0), or dead ahead/behind (== 0, collinear)? Numerically it equals twice the signed area of triangle o-a-b, but "which way do I turn" is the way to think about it. Every routine in this section is that question asked cleverly: segments cross if each one's endpoints lie on opposite *sides* of the other; the hull pops points that make a *right* turn; the shoelace area adds up signed triangles; "is the polygon given clockwise?" is the sign of its area.

```
def cross(o, a, b):
    # cross product of (a - o) and (b - o):
    # > 0 left turn (CCW), < 0 right turn (CW), == 0
collinear
    return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) *
(b[0] - o[0])

def dot(o, a, b):
    return (a[0] - o[0]) * (b[0] - o[0]) + (a[1] - o[1]) *
(b[1] - o[1])

def dist2(a, b):                    # squared distance:
compare these, avoid sqrt
    return (a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2
```

Segments.

```
def on_segment(a, b, p):            # is p on the segment a-
b?
    if cross(a, b, p) != 0:
        return False
    return min(a[0], b[0]) <= p[0] <= max(a[0], b[0]) and \
           min(a[1], b[1]) <= p[1] <= max(a[1], b[1])

def segments_intersect(p1, p2, p3, p4):
    d1 = cross(p3, p4, p1)
    d2 = cross(p3, p4, p2)
    d3 = cross(p1, p2, p3)
    d4 = cross(p1, p2, p4)
    if ((d1 > 0) != (d2 > 0)) and ((d3 > 0) != (d4 > 0)):
        return True                 # proper crossing
    # touching endpoints / collinear overlaps:
    return (on_segment(p3, p4, p1) or on_segment(p3, p4, p2)
or
            on_segment(p1, p2, p3) or on_segment(p1, p2,
p4))

def dist_point_segment(p, a, b):
    if a == b:
        return math.hypot(p[0] - a[0], p[1] - a[1])
    if dot(a, b, p) < 0:            # closest point is
endpoint a
        return math.hypot(p[0] - a[0], p[1] - a[1])
    if dot(b, a, p) < 0:            # closest point is
endpoint b
        return math.hypot(p[0] - b[0], p[1] - b[1])
    return abs(cross(a, b, p)) / math.hypot(b[0] - a[0],
b[1] - a[1])
```

Line intersection point (floats; direction vectors r and s):

```
def line_intersection(p, p2, q, q2):
    rx, ry = p2[0] - p[0], p2[1] - p[1]
    sx, sy = q2[0] - q[0], q2[1] - q[1]
    denom = rx * sy - ry * sx
    if denom == 0:
        return None                 # parallel (or the same
line)
    t = ((q[0] - p[0]) * sy - (q[1] - p[1]) * sx) / denom
    return (p[0] + t * rx, p[1] + t * ry)
```

Polygon area (shoelace) — any simple polygon, vertices in order. Walking the boundary, each edge and the origin form a triangle with a *signed* area; parts outside the polygon are counted once positive and once negative and cancel, leaving exactly the enclosed area:

```
def twice_area(pts):                # keep the doubled area
to stay in integers
    s = 0
    n = len(pts)
    for i in range(n):
        x1, y1 = pts[i]
        x2, y2 = pts[(i + 1) % n]
        s += x1 * y2 - x2 * y1
    return s                        # area = abs(s) / 2;  s
> 0 means CCW order
```

Point in polygon (ray casting). Shoot a ray from the point to the right and count how many polygon edges it crosses: crossing the boundary flips inside↔outside, so an ODD count means inside. Works for any simple polygon, convex or not:

```
def point_in_polygon(p, poly):      # returns 'in', 'out',
or 'boundary'
    inside = False
    n = len(poly)
    for i in range(n):
        a, b = poly[i], poly[(i + 1) % n]
        if on_segment(a, b, p):
            return 'boundary'
        if (a[1] > p[1]) != (b[1] > p[1]):          # edge
crosses the horizontal ray
            x_cross = a[0] + (p[1] - a[1]) * (b[0] - a[0]) /
(b[1] - a[1])
```

```
            if p[0] < x_cross:
                inside = not inside
    return 'in' if inside else 'out'
```

Convex hull — the smallest convex polygon containing all the points ("stretch a rubber band around the nails"). Needed for: farthest pair, "which points are on the outside", minimum enclosing shapes. Andrew's monotone chain builds it in O(n log n): sort points left to right, walk them keeping only left turns (pop whenever the last three make a right turn — the cross product again), once for the lower edge and once for the upper:

```
def convex_hull(pts):
    pts = sorted(set(pts))
    if len(pts) <= 2:
        return pts
    def half_hull(points):
        hull = []
        for p in points:
            while len(hull) >= 2 and cross(hull[-2],
hull[-1], p) <= 0:
                hull.pop()          # <= 0 also removes
collinear points; use < 0 to keep them
            hull.append(p)
        return hull
    lower = half_hull(pts)
    upper = half_hull(pts[::-1])
    return lower[:-1] + upper[:-1]  # each half's last point
starts the other half
# farthest pair of points = farthest pair on the hull (hull
is usually small:
#   brute force over hull pairs is fine)
```

Angles: sort points around a centre with `math.atan2(y - cy, x - cx)`. Compare floats with a tolerance (`abs(x - y) < 1e-9`), never `==`. Clamp arguments of `acos` to [-1, 1].

15. Max flow, matching, SCC

Spot it. Flow problems hide well. Suspect them when things must be *paired or assigned* under conflicts (workers↔tasks, students↔rooms → bipartite matching), when stuff moves through capacities (pipes, roads with limits → max flow), or when you must *cut/block* the cheapest set of edges to separate two places (min cut = max flow). "Mutually reachable" groups in a directed graph → SCC. Variables with "either-or" constraints (x or y must hold) → 2-SAT.

What max flow is. Picture the graph as pipes: every edge has a capacity (litres/second it can carry). Water enters at a *source* s and leaves at a *sink* t. **Max flow** = the most water per second that can travel from s to t using all routes in parallel. The deep fact: this equals the **min cut** — the smallest total capacity of pipes you'd have to sever so that NOTHING reaches t. (Intuition: the flow is limited exactly by its tightest bottleneck, and the bottleneck is a cut.) So "cheapest way to disconnect/separate two things" is the SAME problem as max flow — recognising that is usually the whole difficulty.

How the algorithm works, in one breath: find any s→t path with spare capacity, push water along it, repeat until no such path exists. The one subtlety is the *residual edge*: pushing f units along u→v creates a virtual edge v→u with capacity f, meaning "you may undo this much later" — that's what lets early bad routing fix itself. Dinic's is this plus a speedup (BFS-layer the residual graph, push only along shortest paths).

Dinic's algorithm — max flow = min cut. $O(V^2E)$ worst case, much faster in practice.

```
class Dinic:
    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]

    def add_edge(self, u, v, cap):
        # an edge is [to, capacity_left,
index_of_reverse_edge]
        self.adj[u].append([v, cap, len(self.adj[v])])
        self.adj[v].append([u, 0, len(self.adj[u]) - 1])

    def _bfs(self, s, t):           # layer the residual
graph
        self.level = [-1] * self.n
        self.level[s] = 0
        q = deque([s])
        while q:
            u = q.popleft()
            for v, cap, rev in self.adj[u]:
                if cap > 0 and self.level[v] == -1:
                    self.level[v] = self.level[u] + 1
                    q.append(v)
        return self.level[t] != -1

    def _dfs(self, u, t, pushed):   # push flow along one
path in the layered graph
        if u == t:
            return pushed
        while self.ptr[u] < len(self.adj[u]):
            edge = self.adj[u][self.ptr[u]]
            v, cap, rev = edge
            if cap > 0 and self.level[v] == self.level[u] +
1:
                # edge has spare capacity AND moves one
layer closer to t
                got = self._dfs(v, t, min(pushed, cap))   #
bottleneck so far
                if got > 0:
                    edge[1] -= got                 # use up
capacity ...
                    self.adj[v][rev][1] += got     # ... and
grow the reverse
                    #                                (undo)
edge by the same
                    return got
            self.ptr[u] += 1       # this edge is a dead
end; never retry it
        return 0

    def max_flow(self, s, t):
        flow = 0
        while self._bfs(s, t):
            self.ptr = [0] * self.n
            pushed = self._dfs(s, t, 1 << 60)
            while pushed > 0:
                flow += pushed
                pushed = self._dfs(s, t, 1 << 60)
        return flow
# recursion depth = augmenting path length; fine for a few
thousand nodes
# min-cut edges: after max flow, BFS from s over edges with
cap > 0;
#   cut = original edges from a visited node to an unvisited
node
```

Modelling: several sources/sinks → one super-source/sink with huge-capacity edges. Vertex capacity → split v into v_in → v_out with that capacity. Assignment problems → bipartite graph, source → left (cap 1), right → sink (cap 1).

Why matching is a flow problem. Matching = pair up left things with right things (workers↔tasks, people↔seats), each used at most once, as many pairs as possible. Build: source → every worker with capacity 1 (each works once), worker → every task they can do, every task → sink with capacity 1 (each done once). One unit of flow = one valid pairing, so max flow = max matching.

Bipartite matching (Kuhn's) — the same idea without the flow machinery, O(V·E), less code. Try to place each left vertex in turn; if the right vertex it wants is occupied, recursively try to re-home the occupant to one of ITS other options. If the chain of re-homings succeeds, the matching grows by one.

```
# left vertices 0..nl-1, right vertices 0..nr-1, adj from
left to right
match_right = [-1] * nr             # match_right[v] = left
vertex matched to v

def try_augment(u, visited):
    for v in adj[u]:                # each right vertex u
would accept
        if not visited[v]:
            visited[v] = True       # consider each v once
per augment attempt
            if match_right[v] == -1 or
try_augment(match_right[v], visited):
                # v is free, OR its current partner found
another home
                match_right[v] = u
                return True
    return False                    # u could not be placed
this round

matching = 0
for u in range(nl):
    if try_augment(u, [False] * nr):
```

```
        matching += 1
# König (bipartite): max matching = min vertex cover;
# max independent set = total vertices − max matching
```

Strongly connected components. In a DIRECTED graph, an SCC is a maximal group of nodes that can all reach each other (a tangle of cycles). Why you care: collapse each group to a single dot and the leftover graph has no cycles — it's a DAG, so toposort and DP work on it again. Typical uses: "which tasks are stuck in a dependency loop", and 2-SAT below. Kosaraju's method: one DFS records a finish order; a second DFS on the REVERSED graph, started from the latest finishers, carves out exactly one component per start. (Kosaraju, iterative; components come out numbered in topological order of the collapsed DAG):

```
# g = adjacency list, rg = reversed graph
order = []                          # post-order of a DFS on
g
seen = [False] * n
for s in range(n):
    if seen[s]:
        continue
    seen[s] = True
    stack = [(s, 0)]
    while stack:
        u, i = stack[-1]
        if i < len(g[u]):
            stack[-1] = (u, i + 1)
            v = g[u][i]
            if not seen[v]:
                seen[v] = True
                stack.append((v, 0))
        else:
            order.append(u)
            stack.pop()

comp = [-1] * n                     # comp[v] = component id
c = 0
for s in reversed(order):
    if comp[s] != -1:
        continue
    comp[s] = c
    stack = [s]
    while stack:
        u = stack.pop()
        for v in rg[u]:
            if comp[v] == -1:
                comp[v] = c
                stack.append(v)
    c += 1
```

2-SAT — n true/false variables, constraints all shaped "a OR b" (also encodes "if a then b", "not both", "at least one"). Key move: a clause (a OR b) is two implications — "a false ⇒ b true" and "b false ⇒ a true". Draw every implication as a directed edge between literal-nodes and the logic becomes graph reachability: if x and NOT-x imply each other (land in the same SCC), the formula is impossible; otherwise a valid assignment can be read straight off the component numbering. Encoding: variables $x_0..x_{n-1}$; literal x → node 2x, NOT x → node 2x+1.

```
# clause (a OR b), where na = 1 if a is negated (same for
nb):
def add_clause(a, na, b, nb):
    u = 2 * a + na
    v = 2 * b + nb
    g[u ^ 1].append(v)              # NOT a  implies  b
    g[v ^ 1].append(u)              # NOT b  implies  a
    rg[v].append(u ^ 1)
    rg[u].append(v ^ 1)
# run the SCC above on the 2n literal nodes, then:
# unsatisfiable  iff  comp[2x] == comp[2x + 1] for some x
# assignment: x = True  iff  comp[2x] > comp[2x + 1]
```

16. Toolbox: bitmasks, meet in the middle, testing

Bits from zero. An integer IS a row of bits: `13` in binary is `1101`, meaning 8+4+0+1. Bit k (counting from 0, rightmost) is "worth" 2^k. The operators, each shown on small numbers:

```
1 << 3          # 8  = binary 1000: a one shifted left 3
places (= 2**3)
13 >> 2         # 3  = 1101 shifted right 2 places -> 11
(bottom bits fall off)
12 & 10         # 8  : AND keeps bits set in BOTH      1100
& 1010 = 1000
12 | 10         # 14 : OR  keeps bits set in EITHER    1100
| 1010 = 1110
12 ^ 10         # 6  : XOR keeps bits set in EXACTLY ONE
1100 ^ 1010 = 0110
~x              # NOT: flips every bit; in Python ~x == -x -
1 (use x & ~mask,
                # never bare ~, since Python ints have no
fixed width)
bin(13)         # '0b1101' — print this when debugging bit
code
```

The big idea: an int as a set. Interpret bit i as "item i is in the set". Then `13 = 1101` is the set {0, 2, 3}, and set operations become single machine instructions — this is what "bitmask" means, and why DP over subsets (section 11, TSP) indexes its table by a mask.

```
mask = 0b1101                   # the set {0, 2, 3}
mask >> 2 & 1                   # 1  — is item 2 in the set?
(move bit 2 to the
                                #      bottom, look at it)
mask | (1 << 1)                 # 1111 — ADD item 1    (OR
turns the bit on)
mask & ~(1 << 2)                # 1001 — REMOVE item 2 (AND
with everything-but)
mask ^ (1 << 0)                 # 1100 — TOGGLE item 0 (XOR
flips)
a | b;  a & b                   # union; intersection of two
sets
a & ~b                          # items in a but not in b
bin(mask).count("1")            # 3 — set size (popcount; no
.bit_count() in 3.9)
mask.bit_length()               # 4 — index of the highest
set bit, +1
mask & -mask                    # 1 — keeps ONLY the lowest
set bit (Fenwick uses
                                #     this); mask & (mask-1)
removes it instead
(1 << n) - 1                    # the full set {0..n-1}: n
ones
```

Looping over subsets. The numbers 0 … 2^n–1 ARE all subsets of n items, so one `range` visits every subset:

```
for mask in range(1 << n):          # all 2^n subsets of
items 0..n-1
    chosen = [i for i in range(n) if mask >> i & 1]
    # e.g. n=3: mask 0 -> [], mask 5 = 101 -> [0, 2], mask 7
-> [0, 1, 2]

sub = mask                          # all subsets OF a given
set 'mask'
while sub > 0:                      # (visits each; ~3^n
total over all masks)
    ...
    sub = (sub - 1) & mask          # next smaller number
that fits inside mask
```

Meet in the middle — n ≤ 40: split in two halves of 2^{20} subsets each.

```
def subset_sums(arr):
    sums = [0]
    for x in arr:
        sums = sums + [s + x for s in sums]
    return sums

left = sorted(subset_sums(a[:n // 2]))
answer = 0
for s in subset_sums(a[n // 2:]):
    # count left sums with s + left_sum <= target
    answer += bisect.bisect_right(left, target - s)
```

Stress testing — when WA and you can write a brute force:

```
import random
for test in range(10000):
    n = random.randint(1, 8)
    a = [random.randint(1, 10) for _ in range(n)]
    if fast(a) != brute(a):
        print("MISMATCH on", a, fast(a), brute(a))
        break
else:
    print("all tests passed")
```

Dates. `import datetime`; `datetime.date(2026, 10, 17).weekday()` (Monday = 0); date subtraction gives `.days`; add `datetime.timedelta(days=k)`.

Simulation problems. Read twice, simulate exactly what is written, name variables after the statement, do not optimise prematurely. Most "weird statement" problems are pure careful implementation.